```
Output:
inner()
outer()
```

Hopefully this doesn't need too much explaining. We're generating – and destroying – two local scopes, one within the other. `outer()` is called from global scope, which generates a local scope. `inner()` is called from `outer()` which generates another, separate local scope. Both functions are free to create the same variable identifier; each is protected from the other by their own scope. When execution passes into `inner()` the scope for `outer()` is not affected, it still exists along with its variables.

Remember scope is only destroyed when the boundary is crossed, `outer()` scope persists until the end of the function body: we're free to call `inner()`, have it do its thing, and when control returns to `outer()` PHP just picks up where it left off.

Bear in mind that anonymous functions don't have identifiers like named functions, and that variables used as references to anonymous functions are of course subject to the same scoping rules as any other variable.

Scoping Rules

To conclude this section, let's summarize the scoping rules.

- Variables declared outside a function have global scope.
- Variables declared inside a function have local scope.
- Every function has its own local scope.
- Local scope is generated when execution passes into a function.
- Local scope is destroyed (along with all local variables and data) when exiting a function.
- Locally scoped variables can persist between function calls if they are declared as `static`.

We've spent a lot of time on discussing scope here, with good reason. The scoping rules are also applicable to objects as we shall see soon.